

Parse Server Security Backend

สรุปภาพรวมระบบ Backend สำหรับงานรักษาความปลอดภัยระบบ
Security Alert · Port Scanning · User Quarantine · Audit Log

โปรเจกต์ parse-server-security v1.0.0

ชนิดระบบ Parse Server 7 + MongoDB 7 (Docker Compose)

ภาษา Node.js (CommonJS) — ต้องใช้ Node 18 ขึ้นไป

ที่มาข้อมูล README.md, server.js, cloud/main.js, docker-compose.yml, tests/verify-all.js

วันที่จัดทำ 2 ตุลาคม 2569

วัตถุประสงค์ของรายงาน สรุปสถาปัตยกรรม ความสามารถหลัก มาตรการด้านความปลอดภัย ขั้นตอนการติดตั้ง ขัดทดสอบ และข้อเสนอแนะสำหรับการนำขึ้นใช้งานจริง (Production)

สารบัญ

01 ภาพรวมและจุดประสงค์ของระบบ

02 สถาปัตยกรรมและองค์ประกอบ

03 โครงสร้างไฟล์ในโปรเจกต์

04 ความสามารถหลัก (Cloud Functions)

05 กลไกความปลอดภัย (Security Controls)

06 ขั้นตอนการติดตั้งและใช้งาน

07 ชุดทดสอบและผลการตรวจสอบ

08 การแก้ปัญหาที่พบบ่อย

09 ข้อเสนอแนะสำหรับการใช้งานจริง

10 สรุปภาพรวมและความเสี่ยงที่ต้องระวัง

01 ภาพรวมและจุดประสงค์ของระบบ

ระบบ Backend สำหรับเก็บข้อมูลด้านความปลอดภัย 4 ด้าน บน Parse Server และ MongoDB

โปรเจกต์นี้เป็น Backend สำหรับงาน Security Operations โดยมี 4 หน้าที่หลัก ได้แก่ การจัดการ **Security Alert** การสแกนพอร์ต **Port Scan** การกักกันบัญชีผู้ใช้ที่ถูกบุกรุก (**Quarantine**) และการเก็บ **Audit Log** ที่แก้ไขย้อนหลังไม่ได้

1. Security Alert จัดการเหตุการณ์ความปลอดภัย แบ่งระดับความรุนแรง (critical / high / medium / low) พร้อมสถานะการดำเนินการ (open / in_progress / resolved)	2. Port Scanning สแกนพอร์ตแบบ TCP connect scan เพื่อตรวจจับ service ที่เปิดออกสู่ภายนอกเกินความจำเป็น
3. User Quarantine ล็อกบัญชีผู้ใช้ที่ถูกบุกรุก ยกเลิก session ทั้งหมดทันที และบันทึกเหตุผล	4. Audit Log บันทึกการกระทำทุกรายการลง SecurityLog เขียนได้เฉพาะระบบเท่านั้น (append-only)

หลักการออกแบบที่ยึด

- **Security by Default** — ค่าตั้งต้นเป็นแบบเข้ม (deny) เช่น ปิดการสร้าง Class ใหม่จาก Client และบังคับให้ User เป็น Private
- **Least Privilege** — ฟังก์ชันสำคัญอนุญาตเฉพาะ masterKey หรือสมาชิก Role SecurityAnalyst
- **Immutable Audit** — Audit Log เขียนจาก Cloud Code เท่านั้น Client ทั่วไปเขียนไม่ได้
- **Defense in Depth** — มีทั้งระดับ HTTP (Helmet, CORS), ระดับ Parse Server (rate limit, lockout) และระดับ Data Layer (Cloud Hooks)

7 Cloud Functions	29 กรณีทดสอบ	4 Class หลัก	2 Docker Services	10+ มาตรการด้านความปลอดภัย
----------------------	-----------------	-----------------	----------------------	-------------------------------

02 สถาปัตยกรรมและองค์ประกอบ

สถาปัตยกรรมแบบ 3 ชั้น (Client → Parse Server → MongoDB)



องค์ประกอบหลัก

องค์ประกอบ	พอร์ต	หน้าที่
app (Parse Server)	1337	Express application ที่รวม Parse Server API, Health check และ Parse Dashboard ไว้ใน process เดียว พร้อม graceful shutdown สำหรับ SIGINT/SIGTERM
mongodb	27017	MongoDB 7 เปิด authentication ด้วย username/password และเก็บข้อมูลใน named volume <code>mongodb_data</code>
Cloud Code	—	ตรรกะทางธุรกิจและกฎความปลอดภัยระดับ Data Layer ไหลดจาก <code>cloud/main.js</code>
Parse Dashboard	1337/dashboard	หน้าเว็บจัดการข้อมูล ต้องเข้าสู่ระบบด้วย <code>DASHBOARD_USER</code> / <code>DASHBOARD_PASSWORD</code>

การไหลของ Request

- Client ส่ง request เข้ามาที่ Parse Server พร้อม Header `X-Parse-Application-Id` และ key ที่จำเป็น
- Express ตรวจสอบ Origin ตามรายการ `ALLOWED_ORIGINS` และเพิ่ม Security Headers ผ่าน Helmet
- Parse Server ตรวจสอบ Rate Limit และ Account Lockout ก่อนประมวลผล
- หากเป็น Cloud Function จะเข้าสู่ขั้นตอน `assertSecurityAccess()` เพื่อตรวจสอบสิทธิ์
- ก่อนบันทึกลงฐานข้อมูล Cloud Hooks (`beforeSave` / `beforeFind`) จะบังคับใช้กฎความปลอดภัยอีกชั้นหนึ่ง
- บันทึก `SecurityLog` และตอบกลับผลลัพธ์เป็น JSON

Data Model (Parse Classes)

Class	ฟิลด์หลัก	กฎการเข้าถึง
SecurityAlert	severity, title, details, status, resolutionNote, resolvedAt	Client อ่านได้เฉพาะ alert ที่ tag ถึงตนเอง, เขียนไม่ได้

Class	ฟิลด์หลัก	กฎการเข้าถึง
SecurityScan	target, ports, results, openPorts, severity, status, startedBy	Client อ่าน/เขียนไม่ได้ สร้างผ่าน Cloud Function เท่านั้น
SecurityLog	event, meta, actor	เขียนได้เฉพาะ masterKey (append-only)
_User	accountLocked, lockedAt, lockedReason, securityFlags	แก้ไขได้เฉพาะเจ้าของหรือ masterKey
_Role	SecurityAnalyst	เพิ่มสมาชิกได้เฉพาะ masterKey

03 โครงสร้างไฟล์ในโปรเจกต์

ไฟล์หลักทั้งหมดที่จำเป็นต่อการทำงานของระบบ

ไฟล์ / โฟลเดอร์	หน้าที่
server.js	จุดเริ่มต้นของระบบ ตั้งค่า Express, Helmet, CORS, Parse Server, Health check, Dashboard และ Graceful Shutdown (197 บรรทัด)
cloud/main.js	Cloud Functions ทั้ง 7 ตัว และ Hooks ความปลอดภัยระดับ Data Layer (333 บรรทัด)
docker-compose.yml	กำหนด service <code>app</code> และ <code>mongodb</code> พร้อม healthcheck และ dependency
Dockerfile	สร้าง Docker image ของ Parse Server
.env.example	ต้นแบบตั้งค่า 35 บรรทัด ใช้คัดลอกเป็น <code>.env</code> (ห้าม commit)
.env	ค่าจริงและ Secret ของเครื่อง อยู่ใน <code>.gitignore</code>
tests/verify-all.js	ชุดทดสอบอัตโนมัติ 29 กรณี ครอบคลุมทั้งฟังก์ชันปกติและเคสโจมตี
examples/demo-security.js	ตัวอย่างการเรียก Cloud Functions ที่ละขั้นตอน
api.http.simple	Template REST requests ที่ไม่มี Secret ปลอดภัยสำหรับ Repository
api.http	ไฟล์ local สำหรับ VS Code REST Client (ถูก ignore)
package.json	Dependency 6 ตัว, Node.js ≥ 18 , สคริปต์ start / dev / test / cloud
README.md	คู่มือติดตั้ง ใช้งาน และแก้ปัญหา 313 บรรทัด

Dependency ที่ใช้

Package	Version	การใช้งาน
parse-server	<code>^7.0.0</code>	แกนหลักของ Backend และ API
parse-dashboard	<code>^5.3.0</code>	หน้าเว็บจัดการข้อมูล
express	<code>^4.19.2</code>	HTTP layer สำหรับ Health check และ CORS
helmet	<code>^7.1.0</code>	ตั้งค่า HTTP Security Headers และ CSP
dotenv	<code>^16.4.5</code>	โหลดค่าตั้งค่าจากไฟล์ <code>.env</code>
parse	<code>^4.0.0</code>	Parse JS SDK สำหรับ Client
nodemon	<code>^3.1.0</code>	Auto-reload สำหรับ development

04 ความสามารถหลัก (Cloud Functions)

ฟังก์ชันทั้ง 7 ตัวที่ Client เรียกใช้ผ่าน REST หรือ Parse SDK ได้

ชื่อฟังก์ชัน	สิทธิ์ที่ต้องใช้	หน้าที่
runPortScan	masterKey หรือ Role SecurityAnalyst	สแกนพอร์ตแบบ TCP connect ได้สูงสุด 1,000 พอร์ต ต่อครั้ง บันทึกผลลง SecurityScan แล้วจัดระดับความรุนแรง (0 พอร์ตเปิด = low, 1-4 = medium, 5 ขึ้นไป = high) พร้อมสร้าง Alert อัตโนมัติเมื่อพบพอร์ตเปิด
getSecurityReport	masterKey หรือ Role SecurityAnalyst	สรุปภาพรวมสถานะความปลอดภัย ได้แก่จำนวน Alert แยกตามระดับความรุนแรงและสถานะ จำนวน Critical ที่ยังค้างอยู่ riskLevel และผลสแกน 10 รายการล่าสุด
createSecurityAlert	masterKey หรือ Role SecurityAnalyst	สร้าง Security Alert โดยตรวจสอบระดับความรุนแรงให้อยู่ในชุดที่อนุญาต (critical / high / medium / low)
updateAlertStatus	masterKey หรือ Role SecurityAnalyst	เปลี่ยนสถานะ Alert เป็น open / in_progress / resolved พร้อมบันทึกหมายเหตุและเวลาที่ปิดเรื่อง
quarantineUser	masterKey หรือ Role SecurityAnalyst	ตั้ง accountLocked พร้อมเหตุผลและเวลา ยกเลิก session ทั้งหมดของผู้ใช้ทันที สร้าง Alert และบันทึก Audit Log
unquarantineUser	masterKey หรือ Role SecurityAnalyst	ปลดล็อกบัญชีด้วยการลบฟิลด์ accountLocked , lockedAt และ lockedReason พร้อมบันทึก Audit Log
assignSecurityAnalystRole	masterKey เท่านั้น	เพิ่มผู้ใช้เข้า Role SecurityAnalyst และสร้าง Role พร้อม ACL สาธารณะให้อัตโนมัติหากยังไม่มี

รูปแบบการอนุญาตสิทธิ์แบบสองชั้น

ฟังก์ชันทุกตัวเรียก assertSecurityAccess() เป็นด่านแรก ซึ่งจะอนุญาตเมื่อเป็น masterKey หรือเป็นสมาชิก Role SecurityAnalyst เท่านั้น จากนั้นยังกำหนด requireAnyUserRoles: ['SecurityAnalyst'] ในตัวฟังก์ชันอีกชั้นหนึ่ง ทำให้เกิดการตรวจสอบซ้ำซ้อนซึ่งลดความเสี่ยงจากการตั้งค่าผิดพลาด

ตรรกะการจัดระดับความเสี่ยง (getSecurityReport)

เงื่อนไข	riskLevel ที่ระบบคืนค่า
มี Critical Alert ที่ยังไม่ resolved	CRITICAL
ไม่มี Critical ค้าง แต่มี High Alert	HIGH
ไม่พบ Alert ระดับสูง	NORMAL

05 กลไกความปลอดภัย (Security Controls)

มาตรการป้องกันแบบหลายชั้นตั้งแต่ระดับ HTTP จนถึงระดับฐานข้อมูล

5.1 ชั้น HTTP / Transport

มาตรการ	รายละเอียดการตั้งค่า
Helmet Headers	เพิ่ม Security Headers อัตโนมัติ พร้อมกำหนด Content Security Policy ให้ <code>default-src</code> และ <code>frame-ancestors</code> เป็น <code>'self'</code> (ปิด Cross-Origin Embedder Policy เพื่อรองรับ client แบบ cross-origin)
CORS Whitelist	อนุญาตเฉพาะ Origin ที่ระบุใน <code>ALLOWED_ORIGINS</code> และกำหนด Allowed Headers เฉพาะที่จำเป็นต่อ Parse SDK เท่านั้น
Trust Proxy	เปิด <code>app.set('trust proxy', 1)</code> เพื่อให้ Rate Limit อ่าน IP ผู้ใช้จริงได้ถูกต้องหลังผ่าน reverse proxy
Config Validation	ตรวจสอบ <code>.env</code> ก่อนเริ่ม ถ้าขาด key สำคัญจะหยุดทันที (exit 1) พร้อมเตือนเมื่อยังใช้ค่า default <code>change_me_</code>

5.2 ชั้น Parse Server

มาตรการ	รายละเอียดการตั้งค่า
Rate Limiting	จำกัด 500 requests / 15 นาที / IP (ปรับได้ด้วย <code>RATE_LIMIT_MAX</code>) และเปิด <code>includeInternalRequests: true</code> เพื่อนับ request จาก localhost ด้วย
Account Lockout	ล็อกบัญชีเมื่อ login ผิด 5 ครั้ง เป็นเวลา 5 นาที และปลดล็อกอัตโนมัติเมื่อเปลี่ยนรหัสผ่าน
Master Key IP Restriction	จำกัด <code>masterKey</code> ให้ใช้จาก <code>127.0.0.1, ::1, 172.20.0.1</code> เท่านั้น (ปรับได้ด้วย <code>MASTER_KEY_IPS</code>)
Block Class Injection	<code>allowClientClassCreation: false</code> — Client ไม่สามารถสร้าง Parse Class ใหม่ได้
Private Users	<code>enforcePrivateUsers: true</code> — ผู้ใช้อ่านข้อมูลของผู้อื่นไม่ได้
Protected Fields	ซ่อนฟิลด์ <code>email</code> และ <code>securityFlags</code> ของผู้ใช้ — ต้องใช้ <code>masterKey</code> จึงจะเห็นค่า
Session Revocation	<code>revokeSessionOnPasswordReset: true</code> — เปลี่ยนรหัสผ่านแล้ว session เดิมใช้ไม่ได้
Upload Limit	จำกัดขนาดไฟล์อัปโหลดไว้ที่ 10 MB เพื่อป้องกันการใช้ทรัพยากร
Disable Insecure Adapters	<code>enableInsecureAuthAdapters: false</code> ปิดช่องทางยืนยันตัวตนที่ไม่ปลอดภัย

5.3 ชั้น Data Layer (Cloud Hooks)

Hook	กฎที่บังคับใช้
beforeSave(_User)	สมัครใหม่ได้ / แก้ไขได้เฉพาะเจ้าของหรือ masterKey — ป้องกันการแก้ไขข้อมูลผู้ใช้รายอื่น
beforeLogin	บล็อกการเข้าสู่ระบบของบัญชีที่ถูกกักกัน พร้อมข้อความแนะนำให้ติดต่อ Security Team
beforeSave(SecurityLog)	เขียนได้เฉพาะ masterKey — ทำให้ Audit Log เป็น append-only
afterSave(SecurityLog)	พิมพ์ Audit event ลง console ในรูปแบบ [AUDIT] เพื่อให้ตรวจสอบย้อนหลังได้
beforeSave(SecurityAlert)	สร้างได้เฉพาะผ่าน Cloud Function เท่านั้น Client เขียนตรง ๆ ไม่ได้
beforeFind(SecurityAlert)	ผู้ใช้ทั่วไปเห็นเฉพาะ Alert ที่ tag ถึงตนเอง ส่วน masterKey เห็นทั้งหมด และไม่มีสิทธิ์เลยจะถูกปฏิเสธ
beforeSave(SecurityScan)	เขียนได้เฉพาะ Cloud Code ป้องกันการปลอมผลผลสแกน

5.4 การจัดการ Secret

ข้อควรระวังเรื่อง Secret

ไฟล์ `.env` และ `api.http` ถูกระบุใน `.gitignore` แล้ว และไฟล์ตัวอย่าง `.env.example` ใช้ค่ากลาง `REPLACE_WITH_...` เพื่อป้องกันการสับสน ทั้งนี้ค่าเริ่มต้นของ `MONGO_USERNAME` / `MONGO_PASSWORD` ในไฟล์ตัวอย่างยังเป็นค่าง่าย (`security_user` / `security_pass`) จึงต้องเปลี่ยนก่อนใช้งานจริงเสมอ

06 ขั้นตอนการติดตั้งและใช้งาน

วิธีที่เร็วที่สุดคือใช้ Docker Desktop ร่วมกับ Docker Compose

ขั้นตอนที่ 1 — ดาวน์โหลดโปรเจกต์

```
git clone https://github.com/wattanapat-l-ctrl/parse-server-security.git
cd parse-server-security
```

ขั้นตอนที่ 2 — สร้างไฟล์ตั้งค่า

```
Copy-Item .env.example .env
```

แก้ไขค่าใน `.env` ให้เป็นของจริง โดยเน้น 6 ค่าหลัก:

ตัวแปร	คำอธิบาย
APP_ID	รหัสแอปพลิเคชัน
MASTER_KEY	สิทธิ์สูงสุด — คีย์ที่ต้องรักษาเป็นพิเศษ
JAVASCRIPT_KEY	คีย์สำหรับ Parse JS SDK
REST_API_KEY	คีย์สำหรับ REST request
CLIENT_KEY	คีย์สำหรับ Client application
DASHBOARD_PASSWORD	รหัสผ่านเข้า Parse Dashboard

ขั้นตอนที่ 3 — เริ่มระบบ

```
docker compose up      # รันแบบเห็น log สด
docker compose up -d    # รันแบบ background
```

ครั้งแรก Docker จะ build image ให้อัตโนมัติ ระบบจะเริ่ม MongoDB ก่อนและรอจน healthcheck ผ่าน จึงเริ่ม Parse Server (กำหนดไว้ด้วย `depends_on: condition: service_healthy`) — ไม่ต้องพิมพ์ `npm start` เพิ่ม

ขั้นตอนที่ 4 — ตรวจสอบการทำงาน

```
docker compose ps
Invoke-WebRequest http://localhost:1337/health
```

ทั้ง `mongodb` และ `app` ต้องมีสถานะ `healthy` แล้วเปิด URL เหล่านี้ได้:

URL	หน้าที่
<code>http://localhost:1337/health</code>	Health check คืน <code>{"status":"ok"}</code>
<code>http://localhost:1337/parse</code>	Parse REST API endpoint

URL	หน้าที่
http://localhost:1337/dashboard	Parse Dashboard (ต้องเข้าสู่ระบบ)

คำสั่งที่ใช้บ่อย

ความต้องการ	คำสั่ง
เริ่มระบบ	docker compose up
เริ่มแบบ background	docker compose up -d
ดูสถานะ container	docker compose ps
ดู log ของ Parse Server	docker compose logs -f app
ดู log ของ MongoDB	docker compose logs -f mongodb
หยุดระบบ (เก็บข้อมูลไว้)	docker compose down
หยุดและลบข้อมูลทั้งหมด	docker compose down -v

คำเตือนสำคัญ

คำสั่ง `docker compose down -v` จะ

ลบข้อมูลในฐานข้อมูลทั้งหมด

ควรใช้เฉพาะเมื่อต้องการเริ่มฐานข้อมูลใหม่เท่านั้น นอกจากนี้ชุดทดสอบและไฟล์ตัวอย่างจะสร้างข้อมูลจำนวนมาก จึงไม่ควรรันกับฐานข้อมูล **production**

ทางเลือกสำรอง — รันแบบ Local ด้วย Node.js

```
npm install
docker compose down
docker compose up -d mongodb
npm start
```

ต้องใช้ Node.js 18 ขึ้นไป และห้ามรัน `npm start` พร้อมกับ container `app` เพราะทั้งคู่จะใช้ port 1337 ชนกัน

ใช้งานผ่าน VS Code REST Client

ติดตั้ง Extension ชื่อ **REST Client** แล้วคัดลอกไฟล์ตัวอย่างเป็นไฟล์ local:

```
Copy-Item api.http.simple api.http
```

แก้ค่า `REPLACE_WITH_...` ให้เป็นของจริง แล้วกด **Send Request** ไล่จากบนลงล่าง เพราะบาง request ขึ้นกับผลลัพธ์ของ request ก่อนหน้า (เช่น `alertId` และ `sessionToken`)

07 ชุดทดสอบและผลการตรวจสอบ

ชุดทดสอบอัตโนมัติ 29 กรณี ครอบคลุมทั้งฟังก์ชันปกติและเคสโจเมติ

```
docker compose exec app npm test
docker compose exec app node examples/demo-security.js
```

ผลลัพธ์ที่ถูกต้องคือ **===== สรุป: 29/29 ผ่าน =====** หากมีกรณีใดล้มเหลว สคริปต์จะพิมพ์รายชื่อที่ล้มเหลวและจบด้วย exit code 1

กลุ่มการทดสอบ

#	กรณีทดสอบ	ผลที่คาดหวัง	รายละเอียด
1	GET /health	ผ่าน	ตรวจสอบ health check ที่ root path
2	GET /parse/health	ผ่าน	ตรวจสอบ health check ที่ mount path
3	GET /parse ไม่มี auth	ถูกบล็อก	ต้องได้ 401 หรือ 403 — ยืนยันว่า endpoint ปกป้องกันแล้ว
4	signup ผู้ใช้ A / login / users/me	ผ่าน	ยืนยัน flow พื้นฐานของระบบผู้ใช้
5	runPortScan ด้วย masterKey	ผ่าน	สแกน 6 พอร์ตบน 127.0.0.1 และตรวจสอบผลลัพธ์
6	runPortScan ด้วย session ที่ไม่มี role	ถูกบล็อก	เคสโจเมติ — ยืนยันการบังคับใช้ Role
7	createSecurityAlert → getSecurityReport → updateAlertStatus	ผ่าน	วงจรชีวิตของ Alert ตั้งแต่สร้างจนปิดเรื่อง
8	query SecurityAlert / SecurityScan / SecurityLog	ผ่าน	ตรวจสอบการอ่านข้อมูลด้วย masterKey
9	quarantineUser → session ถูก revoke	ผ่าน	ยืนยันว่า session เดิมใช้ไม่ได้ทันทีหลังกักกัน
10	login บัญชีที่ถูกกักกัน	ถูกบล็อก	เคสโจเมติ — ยืนยันว่า beforeLogin ทำงาน
11	unquarantineUser → login สำเร็จ	ผ่าน	ยืนยันว่าการปลดล็อกทำงานถูกต้อง
12	A แก้ profile ของ B	ถูกบล็อก	เคสโจเมติ — ป้องกันการแก้ไขข้อมูลผู้ใช้รายอื่น
13	A แก้ profile ตัวเอง	ผ่าน	เจ้าของข้อมูลแก้ไขของตัวเองได้
14	query SecurityAlert ด้วย session	ผ่าน	ยืนยันว่า beforeFind กรองข้อมูลตามผู้ใช้
15	client เขียน SecurityLog ตรง ๆ	ถูกบล็อก	เคสโจเมติ — ป้องกันการปลอม Audit Log
16	client สร้าง Class ใหม่	ถูกบล็อก	เคสโจเมติ — ป้องกัน class injection
17	client สร้าง SecurityAlert ตรง ๆ	ถูกบล็อก	เคสโจเมติ — Alert ต้องผ่าน Cloud Function เท่านั้น

#	กรณีทดสอบ	ผลที่คาดหวัง	รายละเอียด
18	Dashboard เข้าได้	ผ่าน	ตรวจสอบหน้าเว็บ Dashboard ตอบกลับ 200
19	<code>.env</code> ไม่มีค่า placeholder	ผ่าน	สแกนหา <code>change_me_</code> ที่เหลืออยู่ในไฟล์ตั้งค่า
20	Rate limit ทำงาน	ผ่าน	เปิด server ย่อยบน port 1338 ด้วย limit 10 แล้วยิง 30 request ต้องได้ 429

หมายเหตุการทดสอบ Rate Limit

การทดสอบนี้จะ spawn server ย่อยขึ้นมาใหม่บน port 1338 พร้อมตั้ง `RATE_LIMIT_MAX=10` แล้วยิง request จนได้สถานะ 429 จากนั้นจึงปิด process ย่อยทิ้ง จึงไม่กระทบ server หลักที่ทำงานบน port 1337

08 การแก้ปัญหาที่พบบ่อย

อาการที่พบบ่อยที่สุดและแนวทางแก้ไข

อาการ	สาเหตุที่เป็นไปได้	แนวทางแก้ไข
<code>RequestError</code> หรือ <code>TcpTestSucceeded: False</code>	VS Code REST Client เชื่อมต่อ port 1337 ไม่ได้	ตรวจสอบว่า Docker ทำงานอยู่ → รัน <code>docker compose ps</code> → ดู log ด้วย <code>docker compose logs --tail 100 app</code> → เปิด <code>http://localhost:1337/health</code> → ถ้า <code>app</code> ไม่ healthy ให้รัน <code>docker compose up -d</code> แล้วรอสักครู่
Compose แจ้งว่าไม่พบ <code>.env</code>	ยังไม่ได้สร้างไฟล์ตั้งค่า	รัน <code>Copy-Item .env.example .env</code> แล้วกรอกค่าให้ครบทุกช่องก่อนเริ่มระบบอีกครั้ง
MongoDB authentication ไม่ผ่าน	ค่าใน <code>.env</code> ไม่ตรงกับฐานข้อมูลเดิมที่ถูกสร้างไว้แล้ว	ตรวจสอบ <code>MONGO_USERNAME</code> , <code>MONGO_PASSWORD</code> และ <code>MONGO_DB</code> — การเปลี่ยนรหัสผ่านใน <code>.env</code> ไม่มีผลกับข้อมูลใน Docker volume เดิม ต้องใช้ <code>docker compose down -v</code> แล้ว <code>docker compose up</code> ใหม่ (ลบข้อมูล)
Port 1337 ถูกใช้งาน	มีโปรแกรมอื่นครองพอร์ต หรือรัน server ซ้ำซ้อน	ตรวจสอบโปรแกรมที่ใช้พอร์ตนี้ หรือเปลี่ยนค่าพอร์ตให้ตรงกันทั้งใน <code>.env</code> และ <code>docker-compose.yml</code>
เตือน <code>[WARN] MASTER_KEY</code> ยังเป็นค่า default	ยังใช้ค่า <code>change_me_...</code> อยู่	แก้ค่าใน <code>.env</code> ให้เป็นคีย์สุ่มยาวก่อน deploy จริง

09 ข้อเสนอแนะสำหรับการใช้งานจริง

รายการสิ่งที่ต้องทำก่อนนำระบบขึ้นใช้งานจริง (Production)

9.1 เรื่องความปลอดภัย

- 1. สร้าง `masterKey` และคีย์อื่น ๆ แบบสุ่มและยาว ห้ามใช้ค่าตัวอย่างจาก GitHub
- 2. เปลี่ยนรหัสผ่านของ MongoDB และ Parse Dashboard
- 3. เปิดใช้งานผ่าน HTTPS และตั้ง `PUBLIC_SERVER_URL` เป็น URL จริง
- 4. จำกัด `ALLOWED_ORIGINS` เฉพาะเว็บไซต์ที่ใช้งานจริง
- 5. จำกัด `masterKeyIps` ให้เหลือเฉพาะเครื่องที่ต้องใช้สิทธิ์ระดับสูง
- 6. เก็บ `.env` ไว้ใน secret manager และไม่ commit ขึ้น Git
- 7. หากเคยเผยแพร่ key หรือรหัสผ่านไปแล้ว ให้เปลี่ยนค่าเหล่านั้นทันที

9.2 ด้านการดำเนินงาน

- 1. สำรองฐานข้อมูล MongoDB ตามรอบเวลาที่กำหนด
- 2. ส่งต่อ Audit Log ไปยังระบบ SIEM หรือ centralized logging เพื่อให้ตรวจสอบย้อนหลังได้ครบถ้วน
- 3. กำหนดนโยบายการทดสอบ — ชุดทดสอบสร้างข้อมูลจำนวนมากจึงห้ามรันกับฐานข้อมูล production
- 4. ทดลองกู้คืนข้อมูลจาก backup เป็นระยะเพื่อยืนยันว่าใช้งานได้จริง

9.3 ข้อเสนอเพิ่มเติม

หัวข้อ	รายละเอียดและเหตุผล
Rate limit แบบหลายระดับ	ปัจจุบันใช้ค่าเดียวทั้งระบบ (500/15 นาที) ควรแยกค่าสำหรับ endpoint ที่อ่อนไหว เช่น <code>/login</code> และ Cloud Functions เพื่อลดผลกระทบข้ามกัน
Hash Password ของ MongoDB	ค่าเริ่มต้นของ Compose ใช้รหัสผ่านง่าย ควรเปลี่ยนเป็นค่าสุ่มทันทีที่ติดตั้งจริง
แยก environment	ควรมีค่าตั้งค่าแยกสำหรับ development, staging และ production ไม่ควรใช้ชุดเดียวกัน
Monitoring	ติดตามจำนวน request ที่ถูกปฏิเสธ (401/403/429) เพื่อตรวจจับการโจมตี
Docker Compose ไม่เหมาะกับ Production	ควรพิจารณาใช้ Orchestrator เช่น Kubernetes หรืออย่างน้อยใช้ Swarm mode เพื่อรองรับ High Availability

9.4 โครงสร้าง Branch

Branch	ไฟล์ที่ตั้งใจให้มี	การใช้งาน
main	โปรเจกต์รวม MongoDB และ Parse Server	ใช้ติดตั้งและใช้งานจริง
feat/app	โค้ดแอป, package, tests และ Dockerfile	ใช้ดูแลส่วนแอป
feat/db	<code>.gitignore</code> และ MongoDB Compose	ใช้ดูแลส่วนฐานข้อมูล

```
git switch main  
git switch -c feat/example  
git add .  
git commit -m "Add example feature"  
git push -u origin feat/example
```

10 สรุปภาพรวมและความเสี่ยงที่ต้องระวัง

ประเมินรวมคุณภาพการออกแบบและประเด็นที่ควรระวัง

จุดแข็งของระบบ

- มาตรการด้านความปลอดภัยครอบคลุมหลายชั้นจริงจัง ทั้งระดับ HTTP, Parse Server config และ Data Layer hooks ไม่ได้พึ่งเพียงการตั้งค่า flag เพียงอย่างเดียว
- ชุดทดสอบครอบคลุมเคสโจมตีจริง ไม่ได้ทดสอบเฉพาะ happy path แต่ตรวจสอบด้วยว่าเส้นทางที่ไม่ควรเปิดถูกปิดจริง
- ระบบ Quarantine ทำงานอย่างสมบูรณ์ ไม่ได้แค่ตั้งธง แต่ยกเลิก session ทั้งหมดและบันทึกเหตุการณ์ไว้ใน Audit Log
- การแยกชั้นการตรวจสอบสิทธิ์ชัดเจน ทั้ง `assertSecurityAccess()` และ `requireAnyUserRoles` ทำงานร่วมกัน ลดความเสี่ยงจากการตั้งค่าพลาด
- เอกสารครบถ้วนภาษาไทย ตั้งแต่การติดตั้ง คำสั่งใช้งาน ไปจนถึงการแก้ปัญหาและคำแนะนำสำหรับ Production

ความเสี่ยงและข้อจำกัด

ระดับ	ประเด็น	ผลกระทบและข้อเสนอ
สูง	การสแกนพอร์ตเป็นการโจมตีของเครือข่ายจริง	ต้องใช้เฉพาะกับระบบที่ได้รับอนุญาตเท่านั้น ควรเพิ่ม allowlist รายการเป้าหมายและจำกัดอัตราการสแกนเพื่อไม่ให้ระบบกลายเป็นเครื่องมือโจมตี
สูง	ค่า default ของ MongoDB และ key ในไฟล์ตัวอย่างยังอ่อน	หาก deploy โดยลืมแก้ <code>.env</code> จะเป็นช่องทางเข้าถึงโดยตรง ควรมีขั้นตอน pre-deploy check ที่บังคับ
ปานกลาง	Rate limit ค่าเดียวทั้งระบบ	อาจทำให้เกินเพียงบ่อยหรือน้อยเกินไปในแต่ละ endpoint ควรแยกค่าตามความอ่อนไหว
ปานกลาง	การสแกนพอร์ตทำแบบ Sequential	หากสแกน 1,000 พอร์ตจะใช้เวลานาน โดยเฉพาะพอร์ตที่ timeout แนะนำให้จำกัดจำนวนพอร์ตหรือทำแบบขนาน
ปานกลาง	Audit Log เก็บในฐานข้อมูลเดียวกับข้อมูลหลัก	หากฐานข้อมูลถูกโจมตี Audit Log จะหายไปด้วย ควรแยกเก็บและส่งต่อแบบ append-only ไปยังที่อื่น
ต่ำ	ใช้ Docker Compose เป็นวิธีหลัก	เหมาะกับ development และงานสาธิต แต่ยังไม่รองรับ High Availability และการ scale แนวนอน
ต่ำ	ยังไม่มีระบบยืนยันอีเมล	<code>verifyUserEmails: false</code> ทำให้สมัครสมาชิกได้โดยไม่ยืนยันตัวตน ควรเปิดใช้งานในระบบจริง

บทสรุป

ระบบ Parse Server Security ที่วิเคราะห์นี้เป็น Backend ที่ออกแบบมาอย่างรอบคอบและนำแนวทาง Security by Default มาใช้จริงอย่างเป็นระบบ การแยกชั้นการตรวจสอบสิทธิ์ทั้งในระดับ Cloud Function และ Cloud Hook ทำให้การบังคับใช้กฎเกิดขึ้นในระดับ Data Layer ซึ่งลดความเสี่ยงจากการตั้งค่าที่ผิดพลาดได้อย่างมีนัยสำคัญ ชุดทดสอบ 29 กรณีครอบคลุมทั้งการทำงานปกติและเคสโจมตี ทำให้มั่นใจได้ว่ามาตรการต่าง ๆ ทำงานจริง

ประเด็นที่ต้องให้ความสำคัญก่อนนำขึ้นใช้งานจริง ได้แก่ การเปลี่ยนค่าความปลอดภัยทั้งหมดในไฟล์ `.env` การบังคับใช้ HTTPS การจำกัด `ALLOWED_ORIGINS` และ `MASTER_KEY_IPS` ให้เหลือเฉพาะที่จำเป็น รวมถึงการกำหนดขอบเขตการใช้งานฟังก์ชันสแกนพอร์ตให้อยู่ภายในระบบที่ได้รับอนุญาตเท่านั้น เมื่อปฏิบัติตามคำแนะนำเหล่านี้แล้ว ระบบนี้มีความพร้อมที่จะใช้งานจริงในระดับองค์กร

รายงานนี้จัดทำจากการวิเคราะห์ซอร์สโค้ดและเอกสารของโปรเจกต์ parse-server-security v1.0.0 (server.js, cloud/main.js, docker-compose.yml, .env.example, tests/verify-all.js, README.md) เอกสารนี้ใช้สำหรับการสรุประบบภายในองค์กร